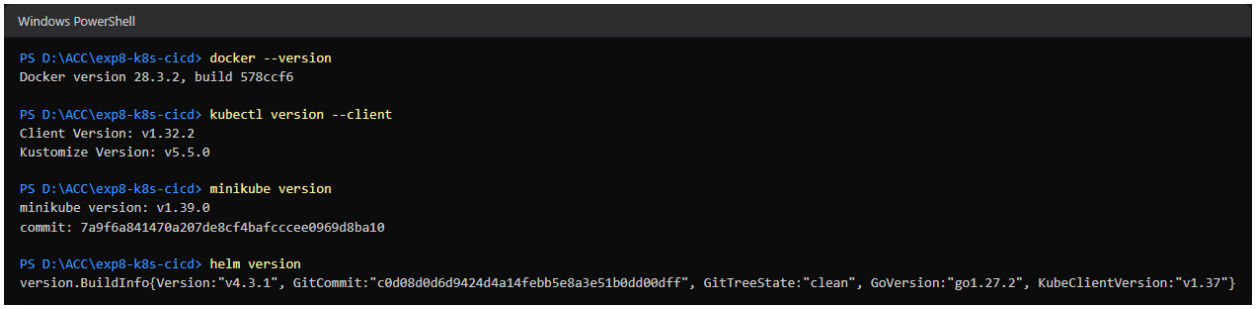
Experiment 8 - Step-by-Step Execution Log

Deploying Applications, CI/CD Pipeline and Cluster Monitoring (Minikube on Windows 11)

Step 1: Setup Step 1-4: Installed Minikube v1.39.0 and Helm v4.3.1 via winget; Docker Desktop 28.3.2 and kubectl were already installed.



Step 2: Setup Step 5: Started Minikube with 'minikube start --driver=docker' and verified with 'kubectl cluster-info' and 'kubectl get nodes' (node Ready).



Step 3: Task 1.1: Created the project repo with app/index.html, Dockerfile and the declarative deployment.yaml (2 replicas, nginx:alpine, CPU/memory requests and limits).

```
Windows PowerShell

PS D:\ACC\exp8-k8s-cicd> tree /F (project files)
./.github/workflows/ci-cd.yml
./.gitignore
./app/index.html
./deployment.yaml
./Dockerfile

PS D:\ACC\exp8-k8s-cicd> type deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-web-app
  labels:
    app: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web-container
          image: nginx:alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
```

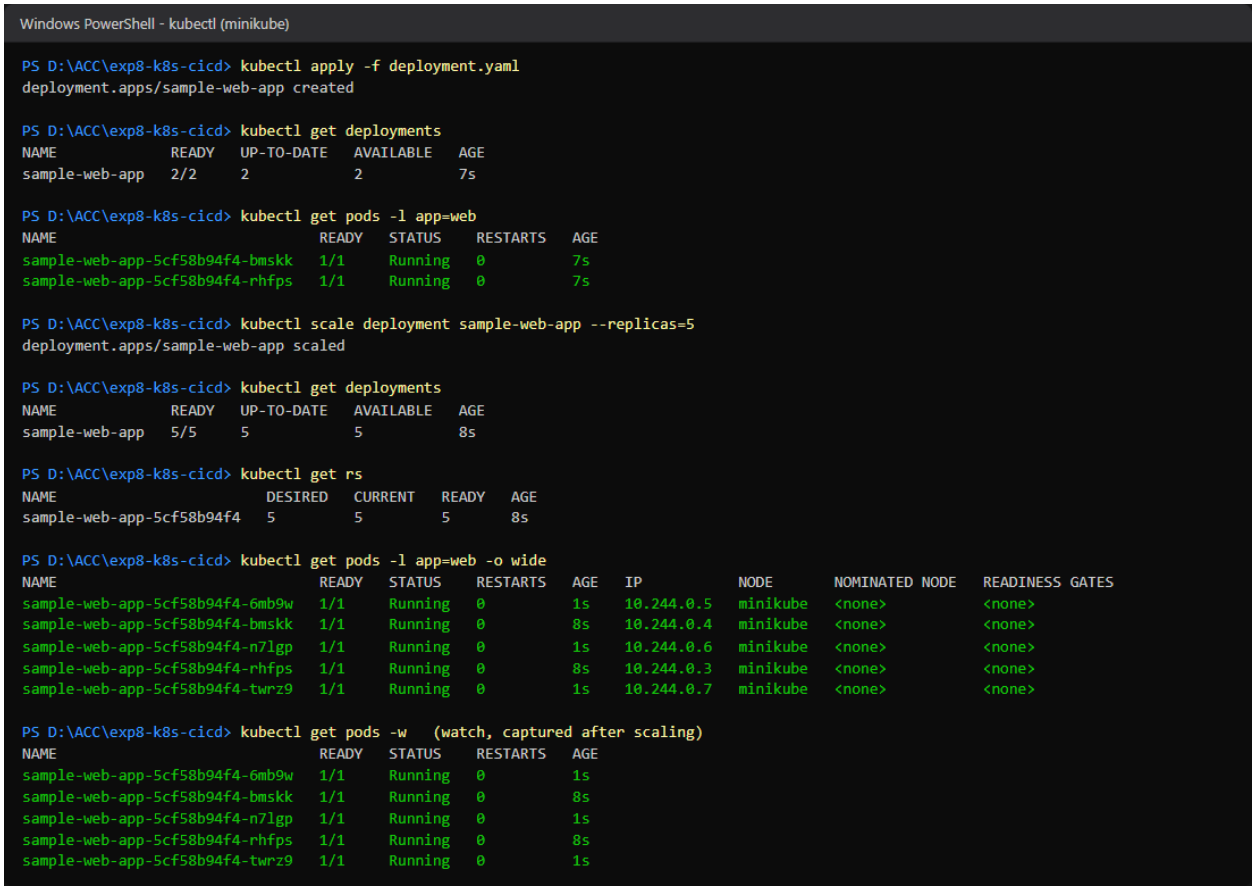
Step 4: Task 2.2: Created .github/workflows/ci-cd.yml (checkout, Docker Hub login from secrets, Buildx, build and push tagged v1.0.<run_number>).

```
Windows PowerShell

PS D:\ACC\exp8-k8s-cicd> type .github\workflows\ci-cd.yml
name: CI/CD Pipeline - Build and Push
on:
  push:
    branches: [ "main" ]
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v3
      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2
      - name: Build and Push Docker Image
        uses: docker/build-push-action@v4
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/sample-web-app:v1.0.${{ github.run_number }}

PS D:\ACC\exp8-k8s-cicd> type Dockerfile
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Step 5: Task 1.2: Applied deployment.yaml, inspected deployments and pods, scaled to 5 replicas; all 5 pods are Running (Screenshot 1 deliverable).



Step 6: Extra check: built the same Docker image locally (docker build), ran it on port 8081 and confirmed the custom page is served - this is exactly what the CI pipeline will build.

Windows PowerShell - docker

```
PS D:\ACC\exp8-k8s-cicd> docker build -t sample-web-app:local .
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 118B 0.0s done
#1 DONE 0.1s

#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 ...

#3 [auth] library/nginx:pull token for registry-1.docker.io
#3 DONE 0.0s

#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 2.5s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 584B 0.0s done
#5 DONE 0.1s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2d33d
#6 resolve docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2d33d 0.0s done
#6 DONE 0.1s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2d33d
#6 sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 0B / 1.40kB 0.2s
#6 sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 1.40kB / 1.40kB 0.2s done
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 0B / 20.53MB 0.3s
#6 sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 0B / 1.21kB 0.2s
#6 sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4 0B / 405B 0.2s
#6 sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 0B / 966B 0.2s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 3.15MB / 20.53MB 0.6s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 6.29MB / 20.53MB 0.8s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 9.44MB / 20.53MB 0.9s
#6 sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 1.21kB / 1.21kB 0.8s done
#6 sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 966B / 966B 0.5s done
#6 sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0 629B / 629B 0.3s done
#6 sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 0B / 4.42MB 0.2s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 16.78MB / 20.53MB 1.2s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 18.87MB / 20.53MB 1.4s
#6 sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 20.53MB / 20.53MB 1.4s done
#6 sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 3.15MB / 4.42MB 0.8s
#6 sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 4.42MB / 4.42MB 0.8s done
#6 extracting sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5
#6 extracting sha256:6711e0c4f5a79e8db637a3412394f29da647e54476d67c5b211a11527568ddc5 0.2s done
#6 DONE 2.0s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:892bb2152464f0c5d8972593a5f5afaf8054552580b699d95f8ff4af08c2d33d
#6 extracting sha256:3bf3671a4e6a18510ca49e3a4a226c2ef734a646e01e34e1ad0eff5faa01e3a0 0.0s done
#6 extracting sha256:b44956913298a55652fa97af34d296339c089250911e46e1a1c0eab8a88538c7 0.0s done
#6 extracting sha256:710ec83ca6cd71fa5636d861343b5660636728f71992fd1b68914e03c7af3ad4 0.0s done
#6 extracting sha256:70683d58fa5c20b5b31c9998a16f6634720d3393dc62934a608754abe449fe3b 0.0s done
#6 extracting sha256:6638b436592c96dd194db76a6b786b60b344d80ebe160e646c871d4e7eff25b5 0.0s done
#6 extracting sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601
#6 extracting sha256:1fdd0614a47ec2896aea36fb280a7b5e8655614dfb56ec1a64002131791de601 0.6s done
#6 DONE 2.6s

#7 [2/2] COPY app/index.html /usr/share/nginx/html/index.html
#7 DONE 0.1s

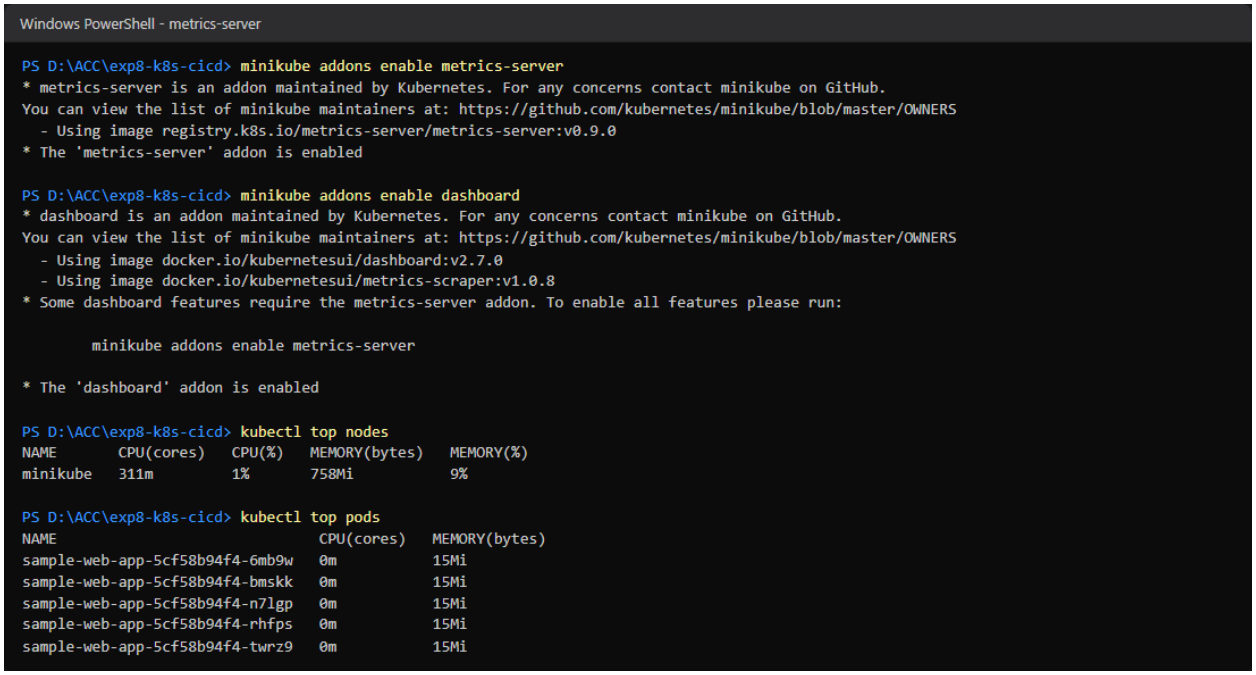
#8 exporting to image
#8 exporting layers 0.1s done
#8 exporting manifest sha256:09a031669c57a3bf5c943da8d95329b3a32d730265e12ba9c011a07f585981df 0.0s done
#8 exporting config sha256:83bf26e23c19313b98256d0cd8c8e0cc2fcf5cae6435f1553bc09193f950e756 done
#8 exporting attestation manifest sha256:5985392e83a38d915b1f87ac26030e97d0f6e7f306764cc65c443d42b6e1c221 0.0s done
#8 exporting manifest list sha256:84ab2a8703fe9546e503f7565e3828857abf6d802584b52339686c16241b137d done
#8 naming to docker.io/library/sample-web-app:local done
#8 unpacking to docker.io/library/sample-web-app:local 0.0s done
#8 DONE 0.2s

PS D:\ACC\exp8-k8s-cicd> docker run -d -p 8081:80 --name exp8-test sample-web-app:local
bb7e344ba5adbe3d4ccbe62580d1581138502927aec6ee4e9c151f4d9fbbd163

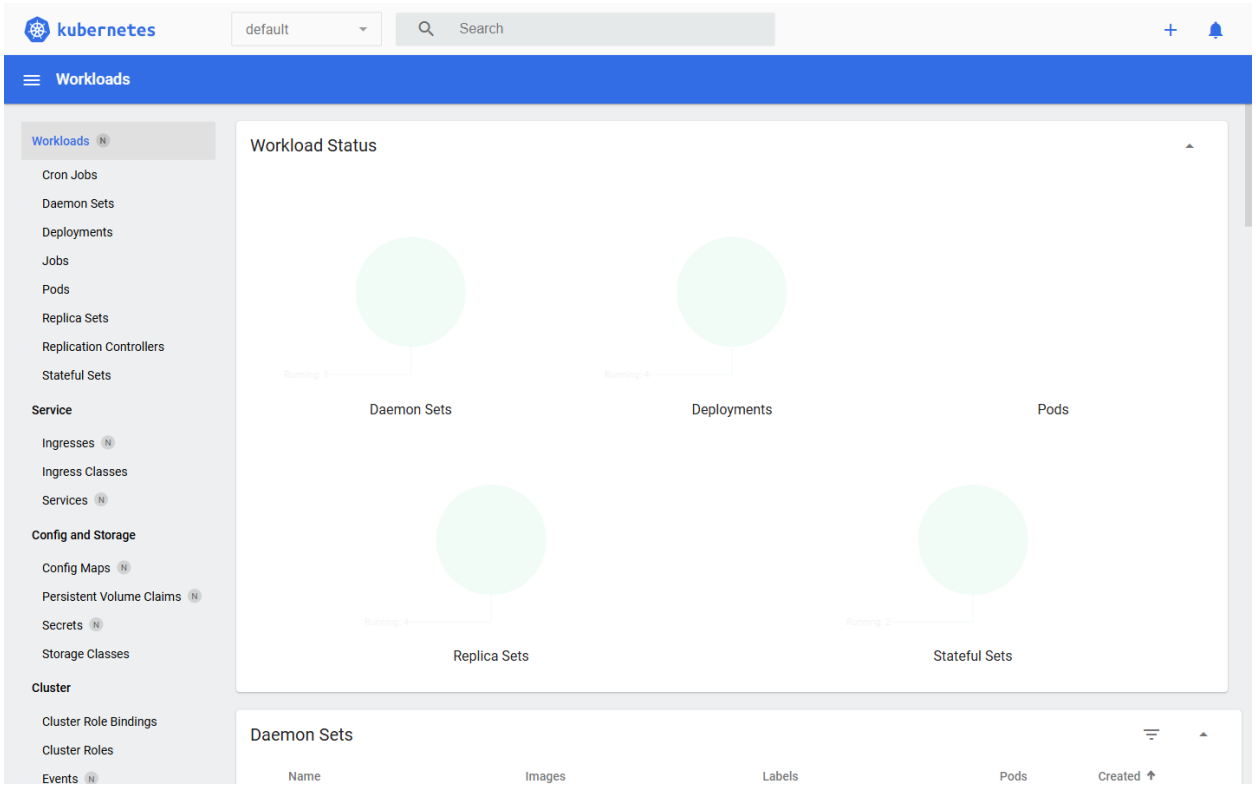
PS D:\ACC\exp8-k8s-cicd> curl http://localhost:8081
<h1>Experiment 8: Kubernetes + CI/CD</h1>

PS D:\ACC\exp8-k8s-cicd> docker images sample-web-app
REPOSITORY       TAG       IMAGE ID       CREATED         SIZE
sample-web-app   local     84ab2a8703fe   5 seconds ago   102MB
```

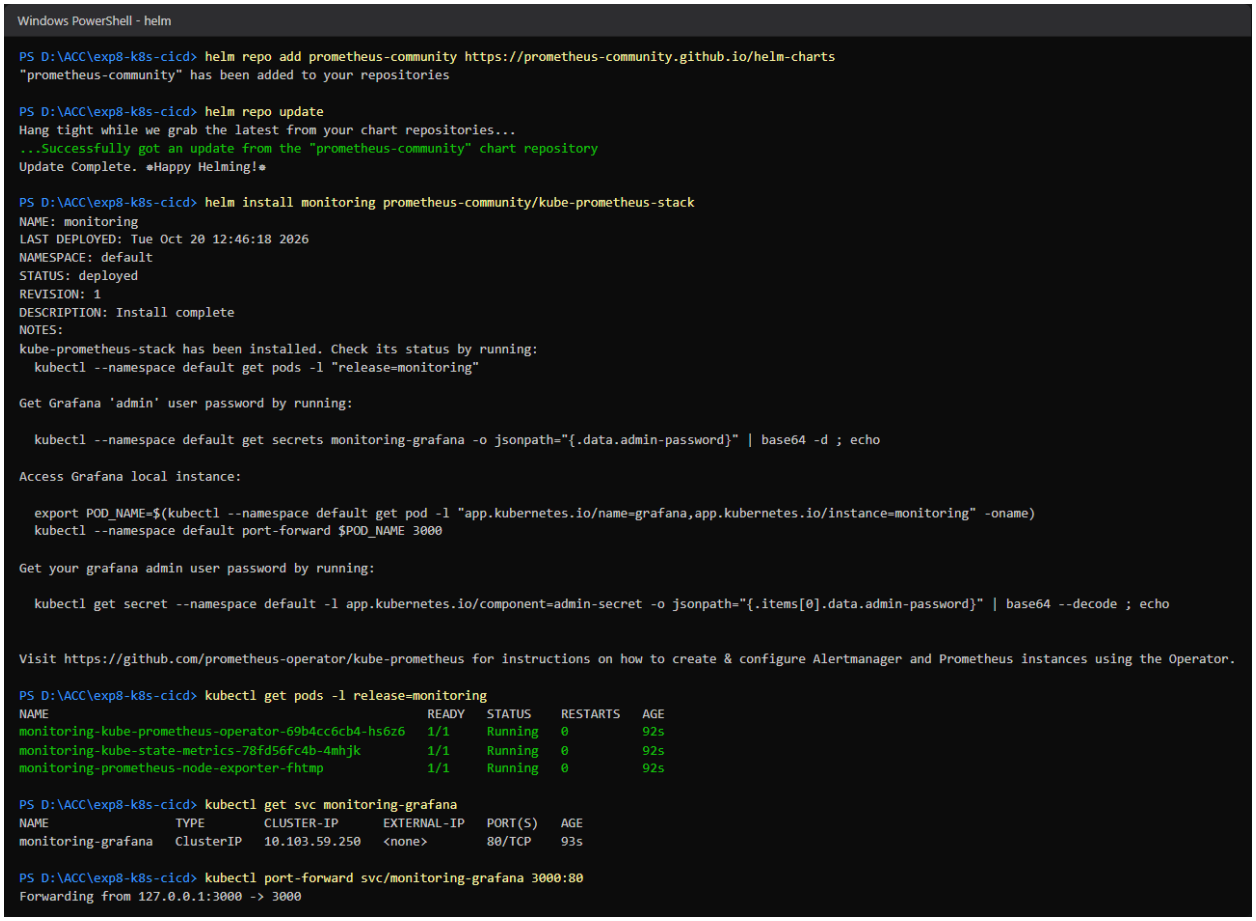
Step 7: Activity 3 Option A: enabled the metrics-server and dashboard addons; 'kubectl top nodes' and 'kubectl top pods' now return live CPU/Memory.



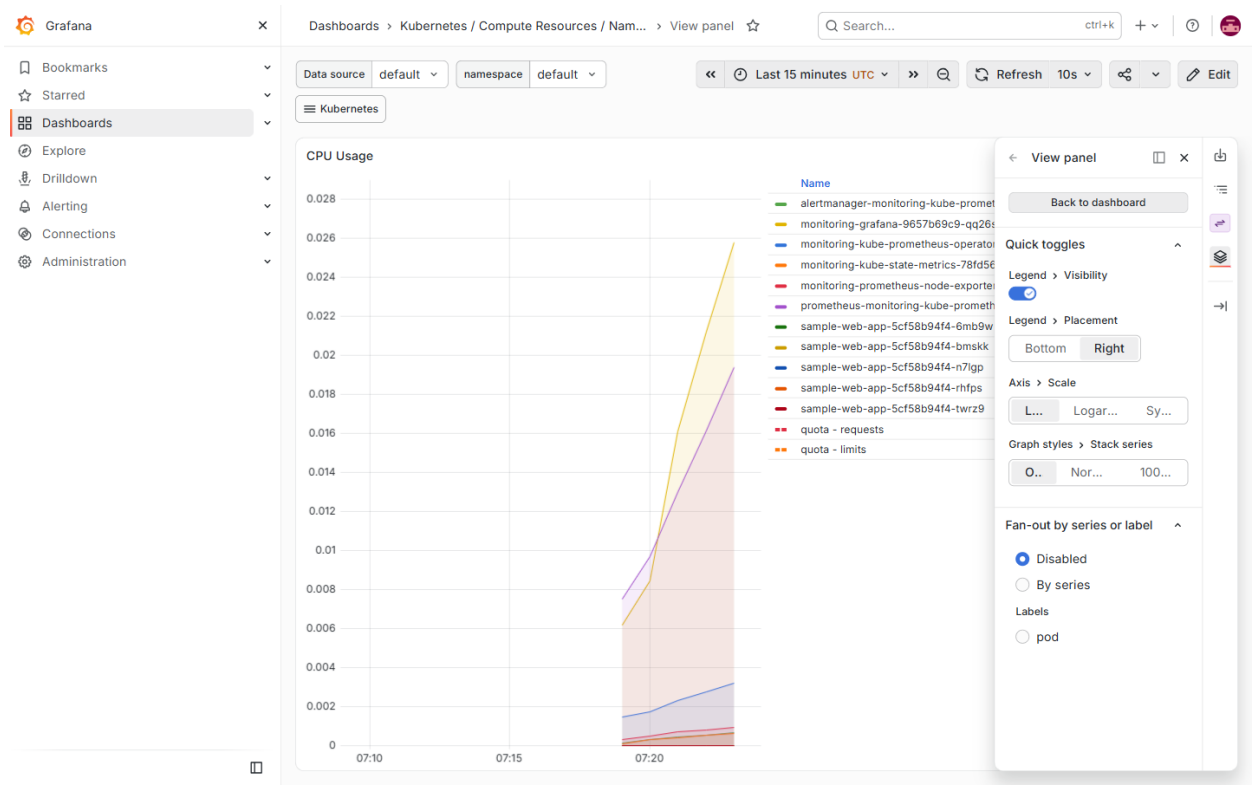
Step 8: Activity 3 Option A: opened the Minikube Kubernetes Dashboard - sample-web-app shows 5/5 pods Running.



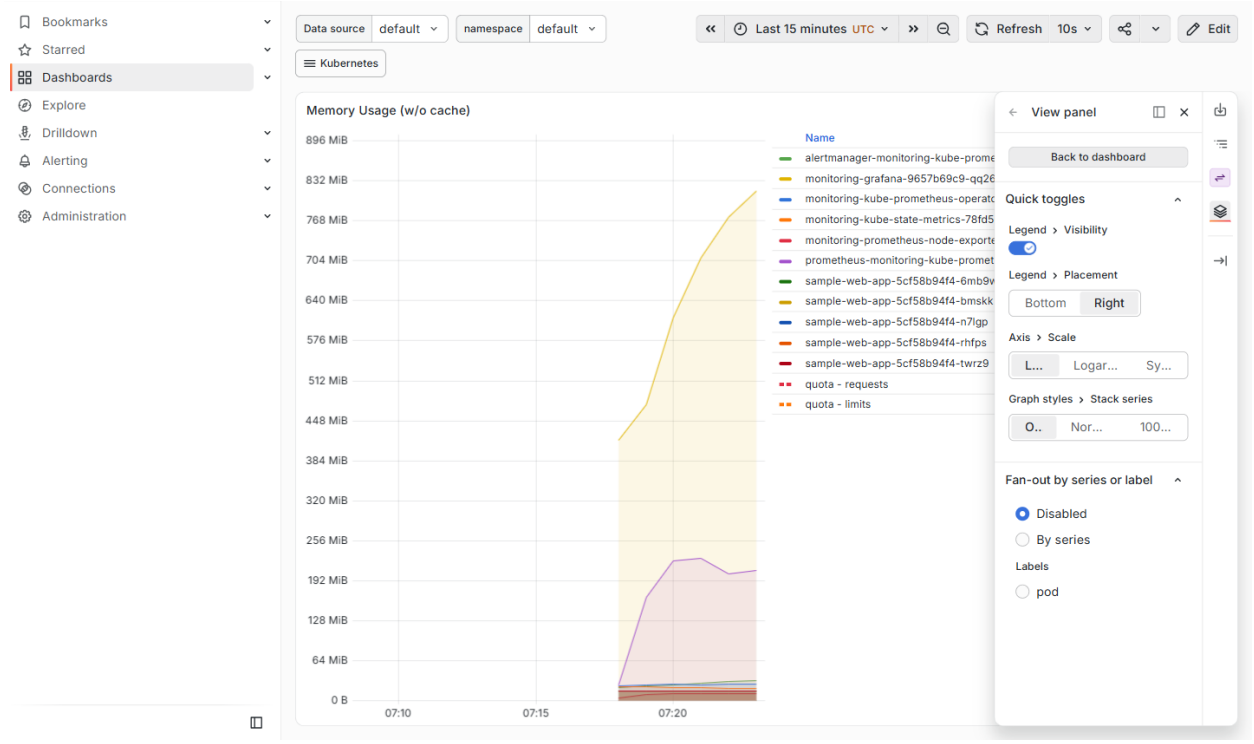
Step 9: Activity 3 Option B: added the prometheus-community Helm repo and installed kube-prometheus-stack; port-forwarded Grafana to localhost:3000.



Step 10: Activity 3 Option B: logged into Grafana (admin / prom-operator) and opened 'Kubernetes / Compute Resources / Namespace (Pods)' - live CPU usage per pod (Screenshot 3 deliverable). A busybox load-generator pod sends traffic to the app service.

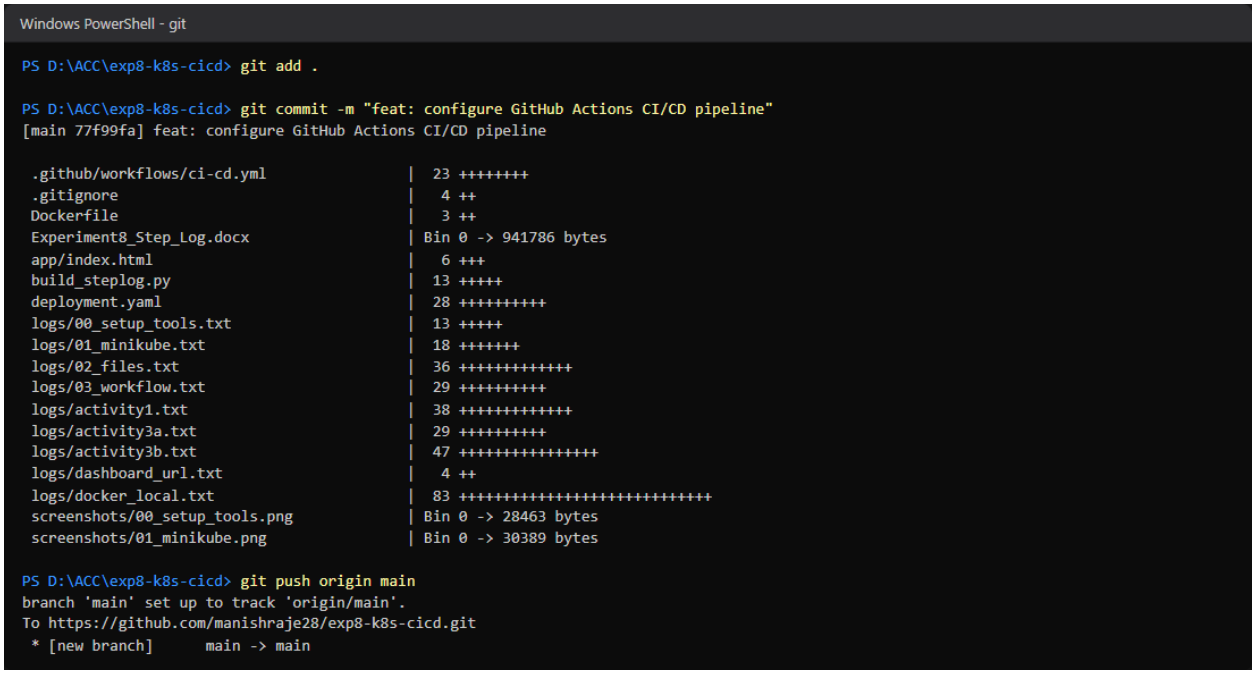


Step 11: Activity 3 Option B: live Memory usage per pod in Grafana, including all 5 sample-web-app replicas.

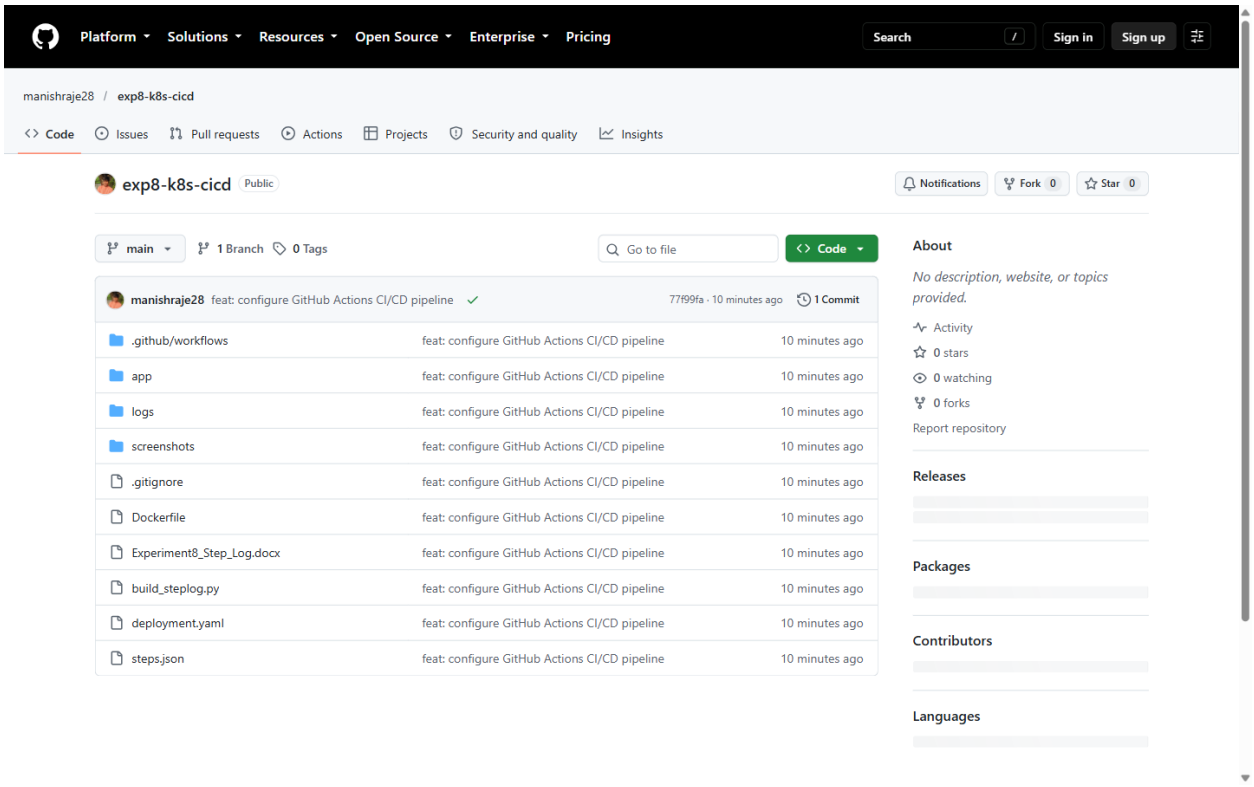


Step 12: Task 2.1 (done by student): created the Docker Hub access token and added the DOCKERHUB_USERNAME and DOCKERHUB_TOKEN repository secrets in GitHub (Settings > Secrets and variables > Actions).

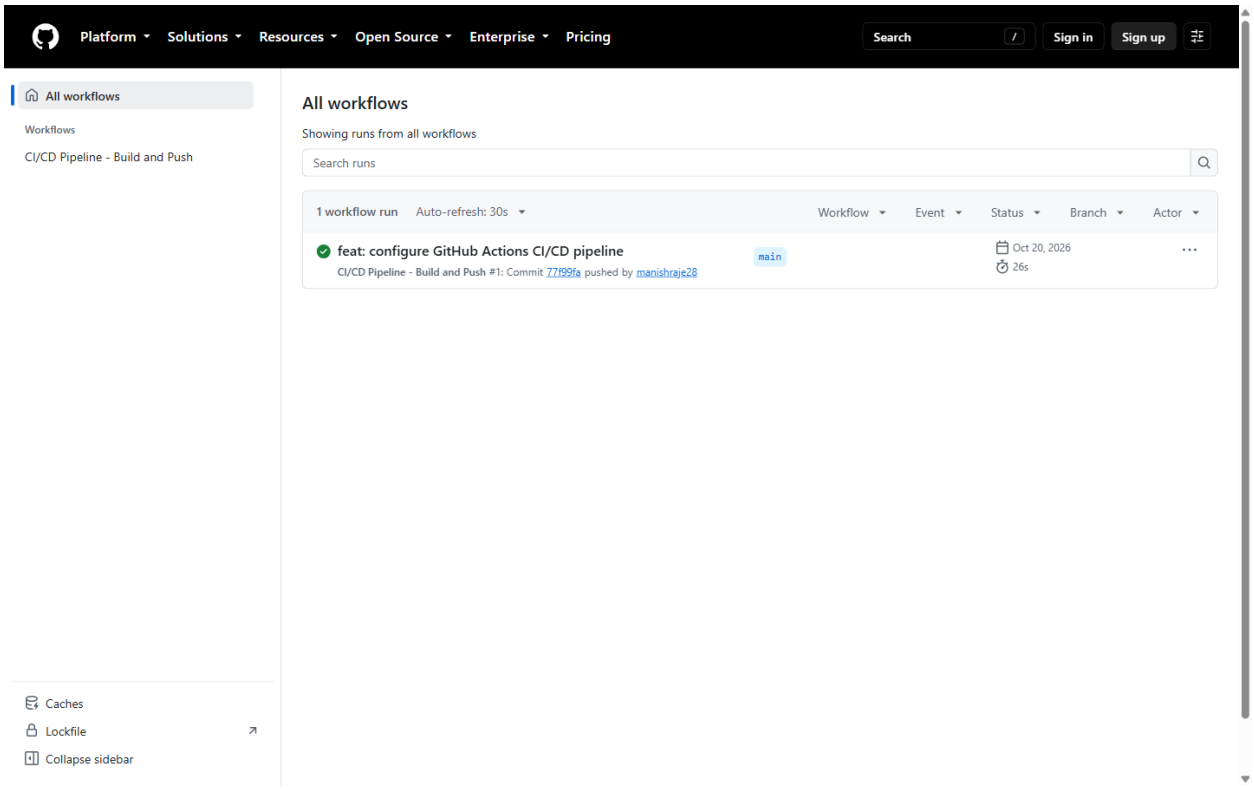
Step 13: Task 2.3: added the GitHub remote, committed with 'feat: configure GitHub Actions CI/CD pipeline' and ran 'git push origin main'.



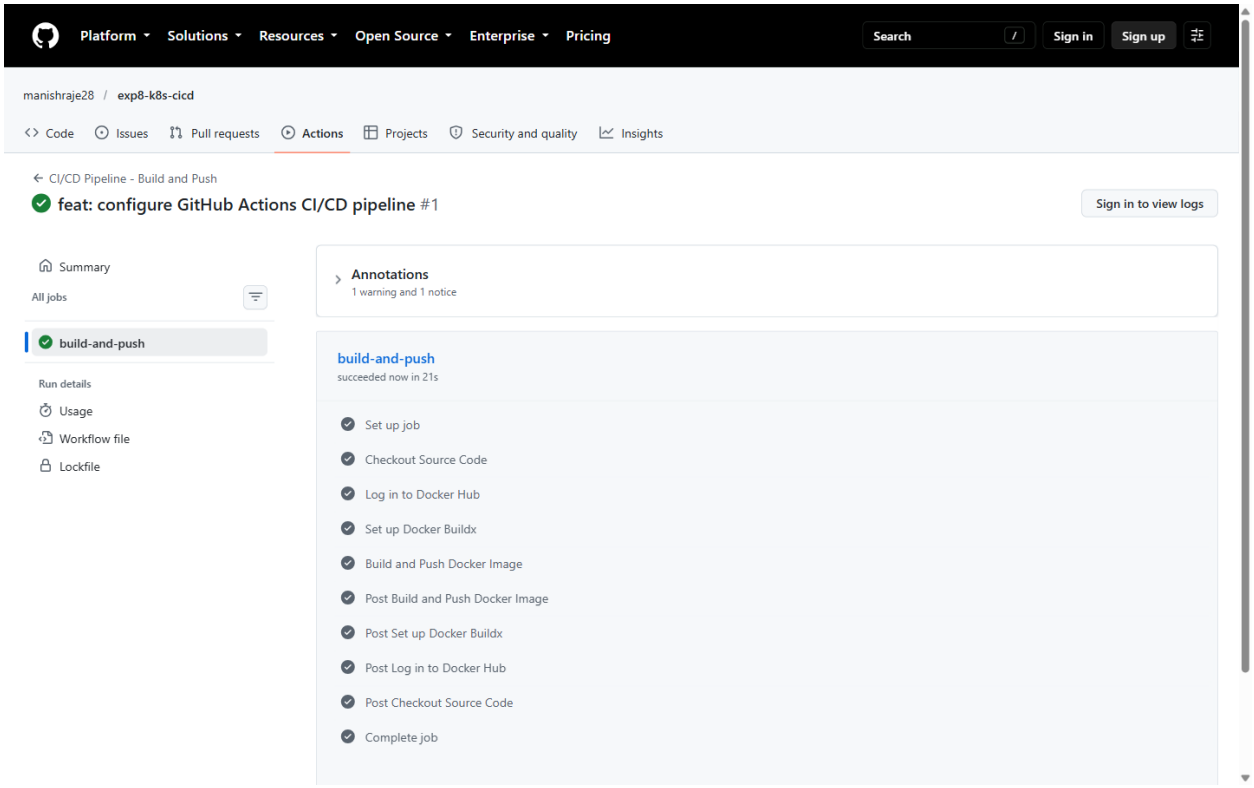
Step 14: Task 2.3: the code is now on GitHub at github.com/manishraje28/exp8-k8s-cicd, with deployment.yaml and .github/workflows/ci-cd.yml.



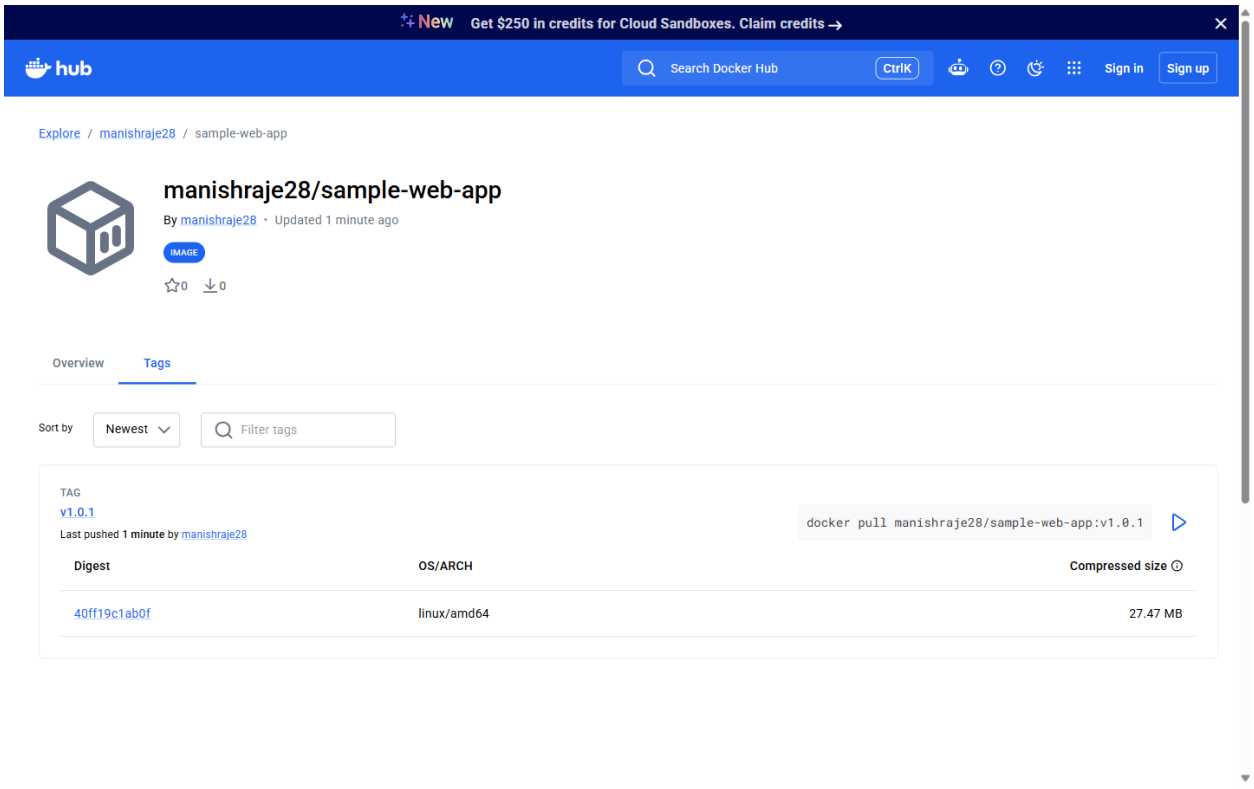
Step 15: Task 2.3: the push triggered the 'CI/CD Pipeline - Build and Push' workflow; run #1 completed green in 26s (Screenshot 2 deliverable).



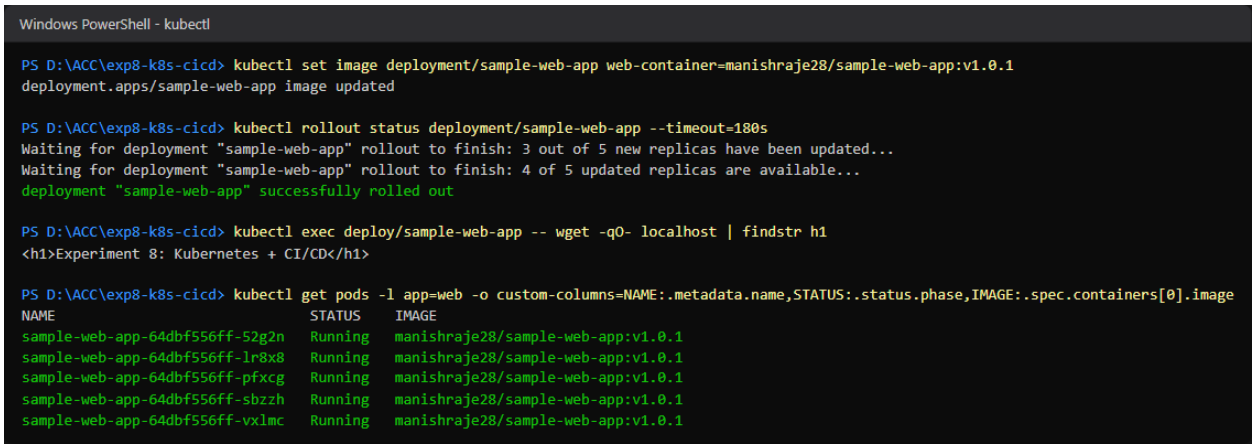
Step 16: Task 2.3: pipeline log - every step of the build-and-push job passed (Checkout, Docker Hub login, Buildx setup, Build and Push).



Step 17: Task 2.3: confirmed image delivery in Docker Hub - manishraje28/sample-web-app:v1.0.1 (tag = v1.0.<run_number>).



Step 18: Extra: rolled the CI-built image into Minikube with 'kubectl set image'; all 5 replicas now run manishraje28/sample-web-app:v1.0.1 and serve the custom page.



Step 19: Phase 3: generated the submission report Experiment8_Report.pdf (repo link, YAML files, pipeline log, the 3 required screenshots, quiz answers, reflection answers), then pushed screenshots and report to GitHub.